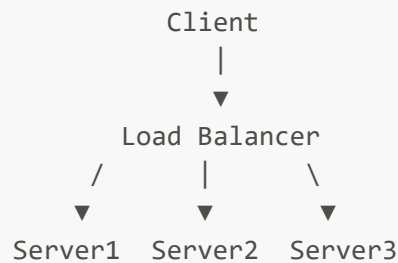


JWT Verification in Distributed Systems

System Architecture

Suppose we have:



A user logs in through **Server1**.

The next request may go to any server.

Question:

How do all servers verify the JWT?

Case 1: Symmetric Key (HS256)

In HS256:

All servers share the same secret key.

```
SECRET_KEY = "mySuperSecret"
```

Used for:

- Signing JWT
 - Verifying JWT
-

Login Flow

Step 1

Client logs in.

```
Client
|
```

```
username/password
|
▼
Server1
```

Step 2

Server1 generates JWT.

Payload:

```
{
  "sub": "42",
  "role": "USER",
  "exp": 1710000900
}
```

Signature:

```
HMACSHA256(
  header.payload,
  SECRET_KEY
)
```

JWT:

```
header.payload.signature
```

Server sends JWT.

API Request

Suppose next request goes to **Server3**.

```
Client
|
Bearer JWT
|
▼
Server3
```

Server3 also has:

```
SECRET_KEY = "mySuperSecret"
```

It computes:

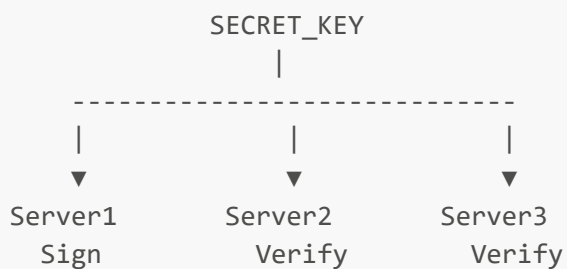
```
HMACSHA256(  
    header.payload,  
    SECRET_KEY  
)
```

Then compares signatures.

If equal:

Authenticated

HS256 Architecture



Advantages

Fast

Only HMAC calculation.

No DB calls.

No external service.

Simple

Easy to implement.

Problems

Key Distribution

Every server needs:

```
SECRET_KEY
```

If you have:

```
100 microservices
```

Then:

```
100 copies of SECRET_KEY
```

exist.

Security Risk

If one service is compromised:

Attacker steals:

```
SECRET_KEY
```

Now attacker can generate valid JWTs.

Example:

```
{
  "sub": "1",
  "role": "ADMIN"
}
```

Sign it.

All servers trust it.

Very dangerous.

Case 2: Asymmetric Keys (RS256)

This is what large systems prefer.

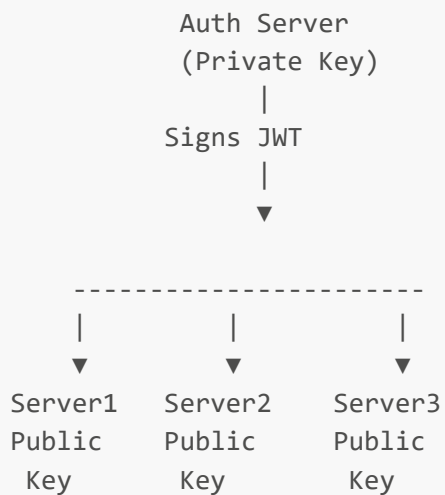
Uses:

- Private Key
- Public Key

Example:

```
Private Key → Auth Service only
Public Key  → Everyone else
```

Architecture



Login Flow

Client sends:

```
username/password
```

↓

Auth Server verifies.

↓

Auth Server signs JWT using:

```
PRIVATE KEY
```

Example:

```
RSASHA256(  
    header.payload,  
    PRIVATE_KEY  
)
```

JWT issued.

API Verification

Suppose request reaches **Server2**.

```
Bearer JWT
```

Server2 verifies using:

```
PUBLIC KEY
```

If valid:

```
Authenticated
```

Important Difference

Symmetric

```
Sign    = SECRET  
Verify  = SECRET
```

Asymmetric

```
Sign    = PRIVATE KEY  
Verify  = PUBLIC KEY
```

Why is RS256 better?

Suppose Payment Service gets hacked.

HS256

Payment Service has:

SECRET_KEY

Attacker can:

✓ Create JWTs

Entire system compromised.

RS256

Payment Service only has:

PUBLIC KEY

Attacker can:

✓ Verify JWTs

but CANNOT:

✗ Generate JWTs

because they don't have:

PRIVATE KEY

Much safer.

Real Example (University Analogy)

Symmetric

Everyone has the principal's stamp.

- Teacher A → can issue certificates
- Teacher B → can issue certificates
- Teacher C → can issue certificates

Anyone can forge degrees.

Asymmetric

Only principal has the official stamp.

Teachers only have sample copies.

- Principal → Issues degree
- Teachers → Verify authenticity

Teachers cannot create degrees.

Public Key Distribution

How do services get the public key?

Auth server exposes:

```
/.well-known/jwks.json
```

JWKS = **JSON Web Key Set**

Flow:

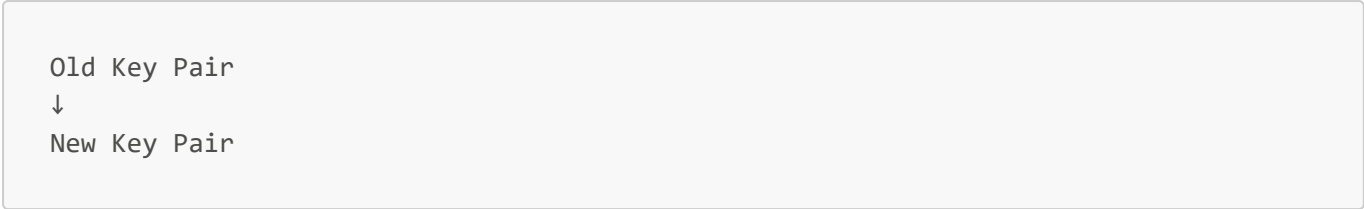
```
graph TD
    AS[Auth Server] --> JWE[JWKS Endpoint]
    JWE --> MS[Microservices download public keys]
```

They cache them.

Key Rotation

Suppose private key leaks.

Generate:



JWT header contains:

```
{
  "alg": "RS256",
  "kid": "key-2026"
}
```

kid means:

Which public key should be used.

Services fetch matching key from JWKS.

This enables seamless key rotation.

System Design Comparison

Feature	HS256 (Symmetric)	RS256 (Asymmetric)
Sign	Secret Key	Private Key
Verify	Secret Key	Public Key
Services can issue JWTs	Yes	No
Services can verify JWTs	Yes	Yes
Key compromise impact	Entire system compromised	Only Auth Server critical
Performance	Faster	Slightly slower
Best for	Small internal systems	Large microservice architectures
Used by	Small apps	OAuth providers, enterprise systems

What Companies Use?

Small Monolith Apps

HS256 is common.

Microservices

RS256 / ES256 is preferred.

Used by:

- Auth0
- Okta
- Google
- Microsoft

Reason:

Thousands of services can verify tokens without ever having the private signing key.

Interview Answer (30 seconds)

With HS256, all services share the same secret key, so any service can both sign and verify JWTs. It's simple and fast but risky because compromising one service compromises the whole system. With RS256, only the authentication service holds the private key and signs tokens, while all other services verify using the public key. This is more secure and is the preferred approach for distributed microservice architectures.